

王道考研——数据结构

WWW.CSKAOYAN.COM

第三章 栈和队列

本节内容

栈(Stack)

基本概念

知识总览

栈(Stack)

定义

基本操作

懶 懶 der ...



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

穿越：线性表的课件

线性表

定义

“逻辑结构”

基本操作

“运算”

懶懶 der...



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

栈的定义

线性表是具有相同数据类型的 n ($n \geq 0$) 个数据元素的有限序列, 其中 n 为表长, 当 $n = 0$ 时线性表是一个空表。若用 L 命名线性表, 则其一般表示为

$$L = (a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

栈 (Stack)是只允许在一端进行插入或删除操作的**线性表**



stack

英 [stæk]

美 [stæk]

n. (整齐的) 一堆; (尤指工厂的) 大烟囱; 堆栈; (干草或谷物的) 堆; 竖着置放的高保真音响设备; 定高分层盘旋 (等待降落) 的机群; (数支步枪支起的锥形) 枪架; (车辆的) 排气管; (浪蚀) 岩柱; 垛, 堆 (木材计量单位)



栈的定义

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表



栈的定义

栈 (Stack) 是只允许在一端进行插入或删除操作的线性表

逻辑结构：与普通线性表相同
数据的运算：插入、删除操作有区别

重要术语：栈顶、栈底、空栈



进栈顺序：

$a1 \rightarrow a2 \rightarrow a3 \rightarrow a4 \rightarrow a5$

出栈顺序：

$a5 \rightarrow a4 \rightarrow a3 \rightarrow a2 \rightarrow a1$

特点：后进先出

Last In First Out (LIFO)

穿越：线性表的基本操作

InitList(&L): 初始化表。构造一个空的线性表L, 分配内存空间。

DestroyList(&L): 销毁操作。销毁线性表, 并释放线性表L所占用的内存空间。

创、销

ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素, 并用e返回删除元素的值。

增、删

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

改、查（“改”之前也要“查”）

其他常用操作:

Length(L): 求表长。返回线性表L的长度, 即L中数据元素的个数。

PrintList(L): 输出操作。按前后顺序输出线性表L的所有元素值。

Empty(L): 判空操作。若L为空表, 则返回true, 否则返回false。

懶 懶 der...



栈的基本操作

InitStack(&S): 初始化栈。构造一个空栈 S, 分配内存空间。

DestroyStack(&S): 销毁栈。销毁并释放栈 S 所占用的内存空间。

创、销

删除栈
顶元素

Push(&S,x): 进栈, 若栈 S 未满, 则将 x 加入使之成为新栈顶。

Pop(&S,&x): 出栈, 若栈 S 非空, 则弹出栈顶元素, 并用 x 返回。

增、删

不删除
栈顶元
素

GetTop(S, &x): 读栈顶元素。若栈 S 非空, 则用 x 返回栈顶元素

查: 栈的使用场景中大多只访问栈顶元素

其他常用操作:

StackEmpty(S): 判断一个栈 S 是否为空。若 S 为空, 则返回 true, 否则返回 false。

懶 懶 der ...



栈的常考题型

进栈顺序:

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$

有哪些合法的出栈顺序?



n 个不同元素进栈，出栈元素不同排列的个数为 $\frac{1}{n+1} C_{2n}^n$ 。

上述公式称为卡特兰（Catalan）数，可采用数学归纳法证明（不要求掌握）。

$$\frac{1}{5+1} C_{10}^5 = \frac{10 * 9 * 8 * 7 * 6}{6 * 5 * 4 * 3 * 2 * 1} = 42$$

知识回顾与重要考点

栈(Stack)

定义

一种操作受限的线性表，只能在栈顶插入、删除

特性：后进先出 (LIFO)

术语：栈顶、栈底、空栈

基本操作

创、销

增、删（元素进栈、出栈，只能在栈顶操作）

查（获得栈顶元素，但不删除）

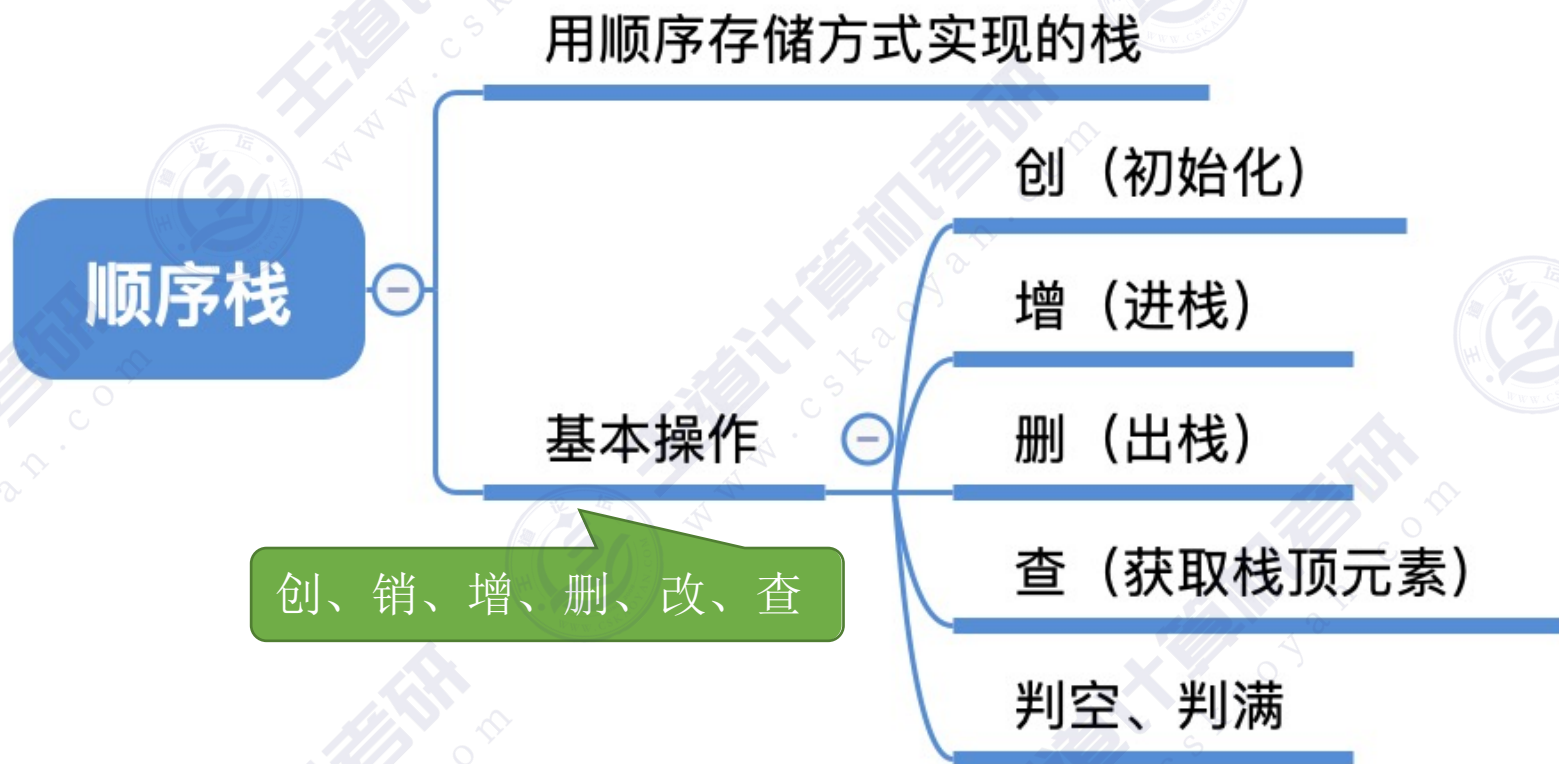
判空



本节内容

顺序栈 的实现

知识总览



顺序栈的定义

```
#define MaxSize 10
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
    int top;
```

```
} SqStack;
```

Sq: sequence —— 顺序

```
void testStack() {
```

```
    → SqStack S; //声明一个顺序栈(分配空间)
```

```
    //...后续操作...
```

```
}
```

//定义栈中元素的最大个数

//静态数组存放栈中元素

//栈顶指针

top = 4

top 指向栈顶元素

顺序存储: 给各个数据元素分配连续的存储空间, 大小为

$\text{MaxSize} * \text{sizeof}(\text{ElemType})$

内存

top(4B)

data[9]

data[8]

data[7]

data[6]

data[5]

e

d

c

b

a

初始化操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
    int top;
```

```
} SqStack;
```

```
//初始化栈
```

```
void InitStack(SqStack &S){
```

```
    S.top = -1;
```

```
}
```

```
void testStack() {
```

```
    SqStack S; //声明一个顺序栈(分配空间)
```

```
    InitStack(S);
```

```
    //...后续操作...
```

```
}
```

```
//定义栈中元素的最大个数
```

```
//静态数组存放栈中元素
```

```
//栈顶指针
```

```
//初始化栈顶指针
```

```
//判断栈空
```

```
bool StackEmpty(SqStack S){
```

```
    if(S.top == -1) //栈空
```

```
        return true;
```

```
    else //不空
```

```
        return false;
```

```
}
```

内存

top = -1

top

data[9]

data[8]

data[7]

data[6]

data[5]

data[4]

data[3]

data[2]

data[1]

data[0]

top

增删改查

进栈操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
    int top;
```

```
} SqStack;
```

//定义栈中元素的最大个数

//静态数组存放栈中元素

//栈顶指针

//新元素入栈

```
bool Push(SqStack &S, ElemType x){
```

```
    if(S.top==MaxSize-1)
```

//栈满，报错

```
        return false;
```

```
    S.top = S.top + 1;
```

//指针先加1

```
    S.data[S.top]=x;
```

//新元素入栈

```
    return true;
```

```
}
```

等价

`S.data[++S.top]=x;`

注意：错误写法！

`S.data[S.top] = x;`

`S.top = S.top + 1;`

`S.data[S.top++]=x;`



内存

top = 1

top

data[9]

data[8]

data[7]

data[6]

data[5]

data[4]

data[3]

data[2]

b

a

top



真的很危险

进栈操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
    int top;
```

```
} SqStack;
```

//定义栈中元素的最大个数

//静态数组存放栈中元素

//栈顶指针

//新元素入栈

```
bool Push(SqStack &S,ElemType x){
```

```
    if(S.top==MaxSize-1)
```

//栈满，报错

```
    return false;
```

```
    S.top = S.top + 1;
```

//指针先加1

```
    S.data[S.top]=x;
```

//新元素入栈

```
    return true;
```

```
}
```

等价

```
S.data[++S.top]=x;
```

内存

top = 9

top

top

k

j

i

h

g

f

e

c

b

a

出栈操作

```
#define MaxSize 10
```

```
typedef struct{
```

```
    ElemType data[MaxSize];
```

```
    int top;
```

```
} SqStack;
```

//定义栈中元素的最大个数

//静态数组存放栈中元素

//栈顶指针

top = 8

top

//出栈操作

```
bool Pop(SqStack &S, ElemType &x){
```

```
    if(S.top == -1)
```

//栈空, 报错

```
        return false;
```

```
    x = S.data[S.top];
```

//栈顶元素先出栈

```
    S.top = S.top - 1;
```

//指针再减1

```
    return true;
```

```
}
```

等价

```
x = S.data[S.top--];
```

注意: 错误写法!

```
S.top = S.top - 1;
```

```
x = S.data[S.top];
```

```
x = S.data[--S.top];
```

内存



数据还残留在内存中, 只是逻辑上被删除了



真的很危险

读栈顶元素操作

```
#define MaxSize 10           //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize];  //静态数组存放栈中元素
    int top;                 //栈顶指针
} SqStack;
```

//出栈操作

```
bool Pop(SqStack &S, ElemType &x){
    if(S.top == -1)           //栈空, 报错
        return false;
    x = S.data[S.top--];      //先出栈, 指针再减1
    return true;
}
```

唯一区别

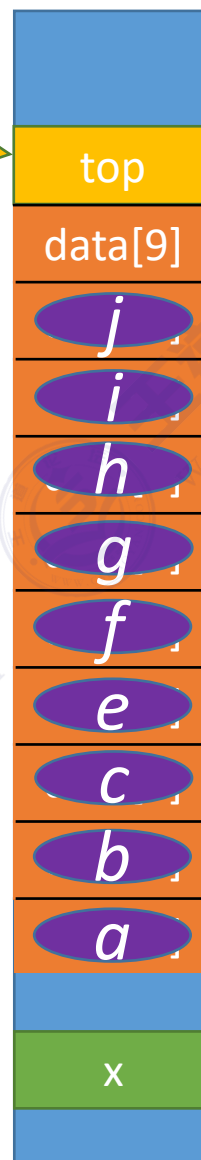
//读栈顶元素

```
bool GetTop(SqStack S, ElemType &x){
    if(S.top == -1)           //栈空, 报错
        return false;
    x = S.data[S.top];        //x记录栈顶元素
    return true;
}
```

内存

top = 8

top →



另一种方式

```
#define MaxSize 10           //定义栈中元素的最大个数
typedef struct{
    ElemType data[MaxSize];  //静态数组存放栈中元素
    int top;                 //栈顶指针
} SqStack;
```

//初始化栈

```
void InitStack(SqStack &S){
    S.top=0;
}
```

//静态数组存放栈中元素
//栈顶指针

//初始化栈顶指针

```
void testStack() {
    SqStack S; //声明一个顺序栈(分配空间)
    InitStack(S);
    //...后续操作...
}
```

//判断栈空

```
bool StackEmpty(SqStack S){
    if(S.top==0) //栈空
        return true;
    else //不空
        return false;
}
```

内存

top = 0

top(4B)

data[9]

data[8]

data[7]

data[6]

data[5]

data[4]

data[3]

data[2]

data[1]

data[0]

空栈

top

另一种方式

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int top;
} SqStack;
```

//定义栈中元素的最大个数
//静态数组存放栈中元素
//栈顶指针

等价

$S.data[S.top] = x;$
 $S.top = S.top + 1;$

$S.data[S.top++] = x;$

进栈

$S.top = S.top - 1;$
 $x = S.data[S.top];$

$x = S.data[--S.top];$

出栈

栈满的条件: $top == MaxSize$

顺序栈的缺点:
栈的大小不可变

注意审题 啊喂!

题目不对劲

top = 5

top 指向下一个
可以插入的位置

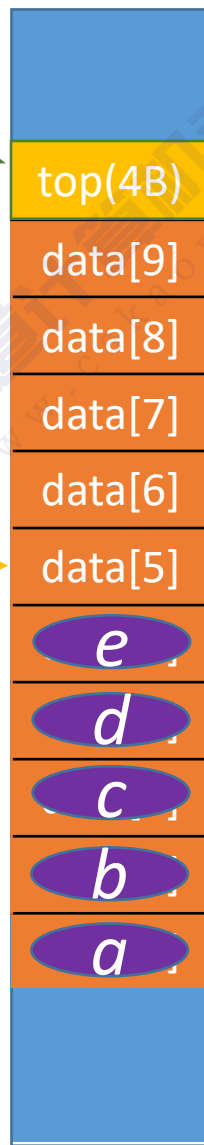
top



不会犯错



内存



共享栈

两个栈共享同一片空间

```
#define MaxSize 10
typedef struct{
    ElemType data[MaxSize];
    int top0;
    int top1;
} ShStack;
```

```
//初始化栈
void InitStack(ShStack &S){
    S.top0=-1;
    S.top1=MaxSize;
}
```

//定义栈中元素的最大个数

//静态数组存放栈中元素

//0号栈栈顶指针

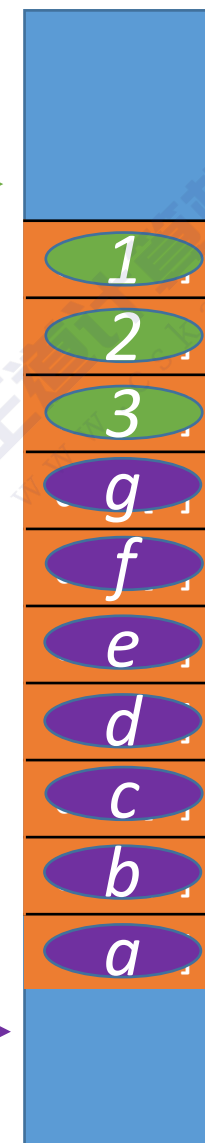
//1号栈栈顶指针

//初始化栈顶指针

top1 →

top0 →

内存



栈满的条件: $top0 + 1 == top1$

知识回顾与重要考点

top=-1

销? —— 清空、回收

```
void testStack() {  
    SqStack S;  
    InitStack(S);  
    //...后续操作...  
}
```

声明栈时
分配内存

函数运行结束后系
统自动回收内存

不会犯错



顺序栈

顺序存储，用静态数组实现，并需要记录栈顶指针

基本操作 创、增、删、查

都是 O(1) 时间复杂度

两种实现

初始化时 top=-1

入栈

$S.data[++S.top] = x;$

出栈

$x = S.data[S.top--];$

获得栈顶元素

$x = S.data[S.top];$

栈空/栈满条件是?

初始化时 top=0

入栈

$S.data[S.top++] = x;$

出栈

$x = S.data[--S.top];$

获得栈顶元素

$x = S.data[S.top-1];$

栈空/栈满条件是?

两个栈共享同一片内存空间，两个栈从两边往中间增长

共享栈

初始化

0号栈栈顶指针初始时 $top0 = -1$; 1号栈栈顶指针初始时 $top1 = MaxSize$;

栈满条件

$top0 + 1 == top1$;

本节内容

链栈 的实现

知识总览

用链式存储方式实现的栈

链栈

基本操作

创（初始化）

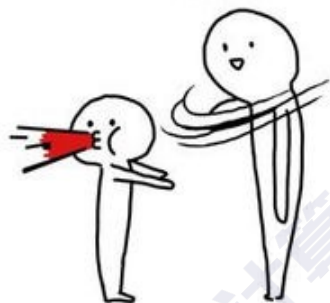
增（进栈）

删（出栈）

查（获取栈顶元素）

判空、判满

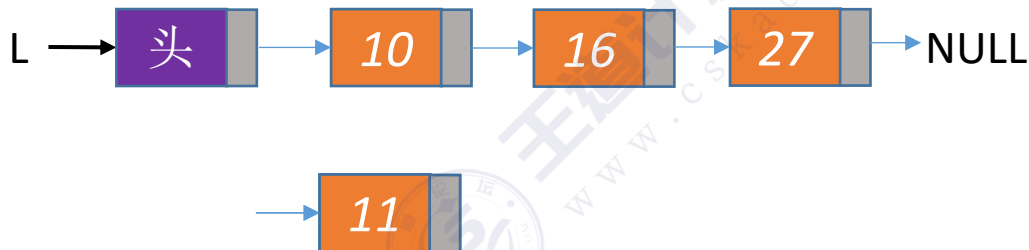
叫你偷懒



穿越：头插法建立单链表

对头结点的
后插操作

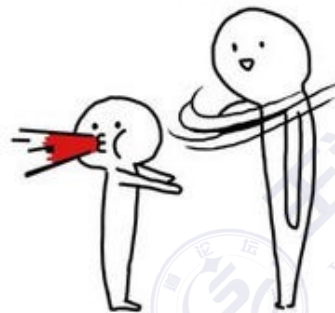
对应：进栈



//后插操作：在p结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){  
    if (p==NULL)  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    if (s==NULL) //内存分配失败  
        return false;  
    s->data = e; //用结点s保存数据元素e  
    s->next=p->next;  
    p->next=s; //将结点s连到p之后  
    return true;  
}
```

叫你偷懒



头插法建立单链表：

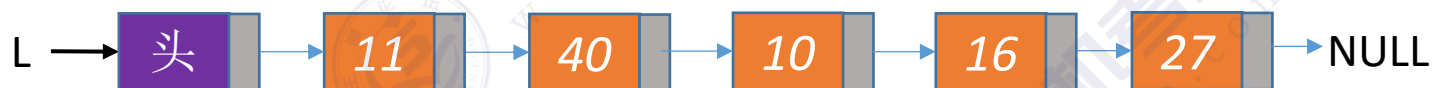
初始化单链表

```
While 循环 {  
    每次取一个数据元素 e;  
    InsertNextNode (L, e);  
}
```

穿越：单链表的删除操作

对应：出栈

对头结点的
“后删”操作



又偷懒!!



链栈的定义

```
typedef struct Linknode{  
    ElemType data;           //数据域  
    struct Linknode *next;   //指针域  
} *LiStack;                  //栈类型定义
```

带头结点的初始化



进栈/出栈都只能在栈顶一端进行
(链头作为栈顶)

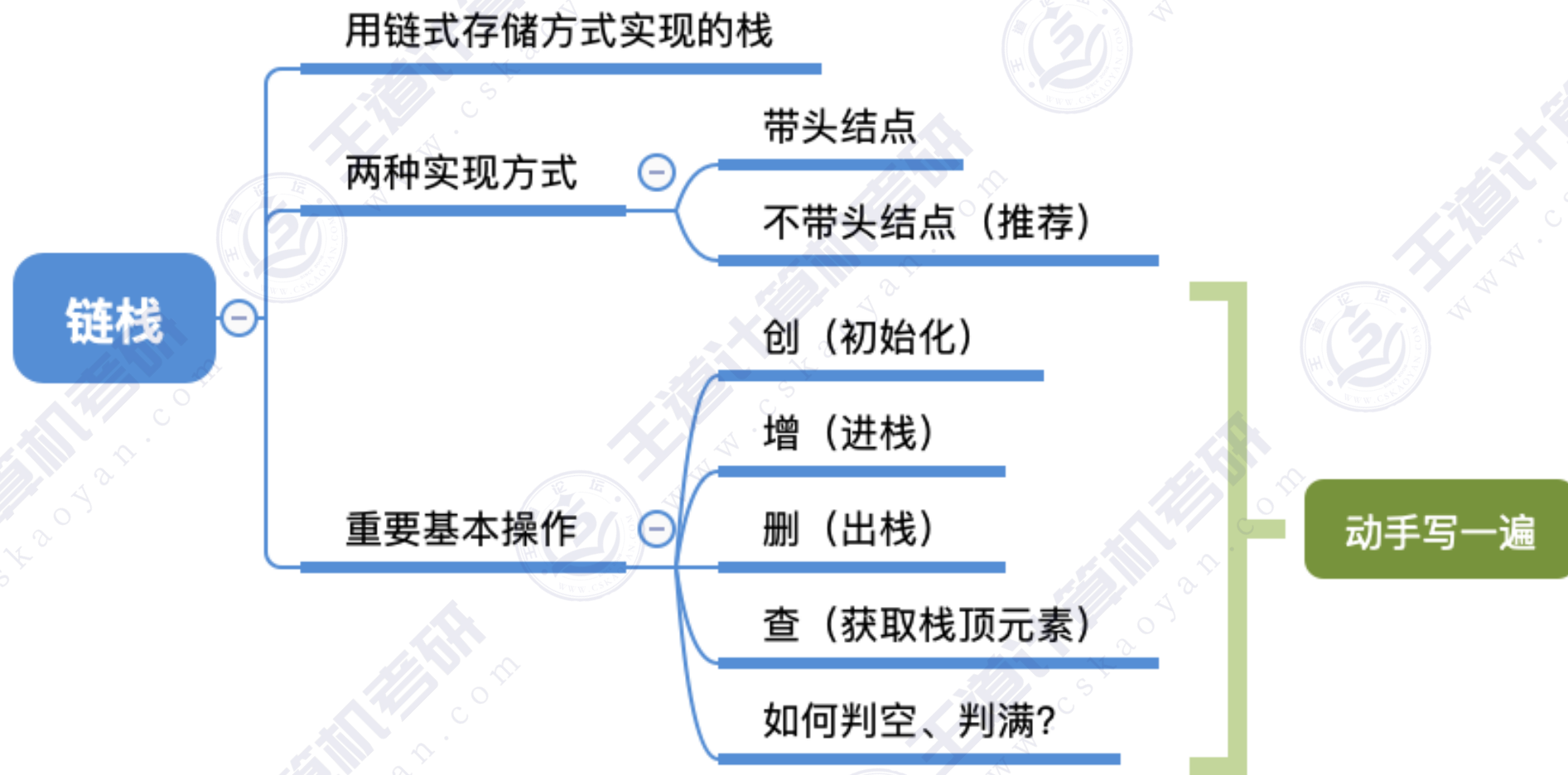


S → NULL

不带头结点的初始化



知识回顾与重要考点



小建议



考试比的是“输出”的能力！



一根直肠通大脑
学到多少拉多少